I Wanna Be Like You

Robust & Rapid Emulation for Efficient Trial Design

Jack Fraser-Govil

23/03/26

Why do we want to Emulate?

Why do we want to Emulate?

- Suppose we have some computational model of the effect of an intervention

$$\boldsymbol{y} = \hat{M}(\boldsymbol{\pi})$$

Why do we want to Emulate?

- Suppose we have some computational model of the effect of an intervention

$$\boldsymbol{y} = \hat{M}(\boldsymbol{\pi})$$

- Then suppose we wish to compute some complex quantity - such as the predictive power

$$p = \sum_{\text{permutations}} \sum_{\boldsymbol{\pi}} f(\boldsymbol{y}(\boldsymbol{\pi}))$$

Why do we want to Emulate?

- Suppose we have some computational model of the effect of an intervention

$$\boldsymbol{y} = \hat{M}(\boldsymbol{\pi})$$

- Then suppose we wish to compute some complex quantity - such as the predictive power

$$p = \sum_{\text{permutations}} \sum_{\boldsymbol{\pi}} f(\boldsymbol{y}(\boldsymbol{\pi}))$$

The problem

If $\hat{M}$ is slow (minutes-to-hours), it will take years-to-decades to compute p.

Why do we want to Emulate?

What is an Emulator?

An emulator is a function $\hat{E}_M$ which *behaves* like $\hat{M}$, but which is vastly quicker to compute.

Why do we want to Emulate?

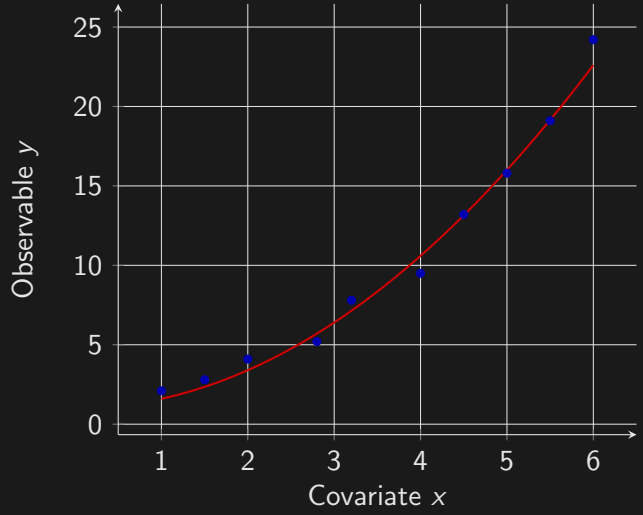
What is an Emulator?

An emulator is a function $\hat{E}_M$ which *behaves* like $\hat{M}$, but which is vastly quicker to compute.

I wanna walk like you, talk like you...

Estimators, Regression

This is not a new idea!



Machine Learning

In recent years[1], we can use machine learning

- Vast non-parametric models

[1] Though even this is becoming 'old'

Machine Learning

In recent years[1], we can use machine learning

- Vast non-parametric models
- Condition it on observable data

Now give me the secret, mancub
C'mon, clue me what to do

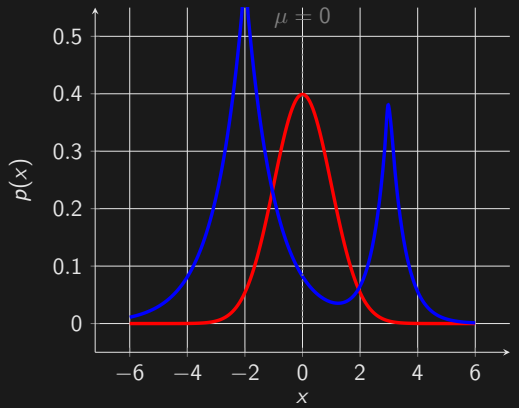
- Hope it learns the correct behaviour

[1] Though even this is becoming 'old'

The Limitation

Distributions, not means

If $\hat{M}$ is stochastic, then $\boldsymbol{y}$ is not a number - it is a *realisation on a random process*. These methods serve only to predict the mean, $\mathbb{E}(\boldsymbol{y})$, not the distributions around that point.



Machine Learning (Again)

There *are* ML models which give distributions (Bayesian Neural Networks), but we may choose to avoid them:

1. Require lots of data (expensive - recall computing $\hat{M}$ is the bottleneck)
2. Black boxes (aesthetically/scientifically dubious)

Gaussian Processes

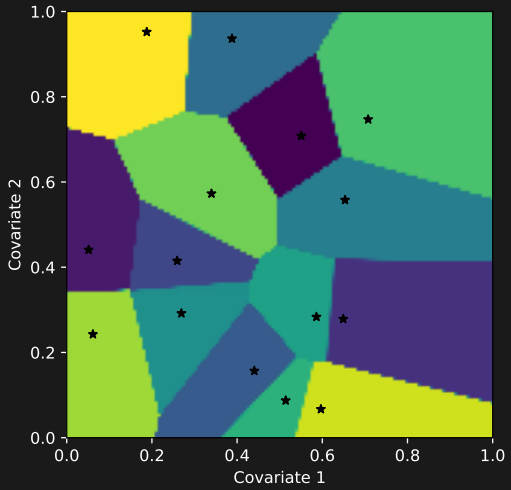
A modelling process where one assumes

$$\boldsymbol{y}(\boldsymbol{\pi}) \sim \mathcal{N}\left(\sum_i w_i(\pi)\boldsymbol{y}_i, \Sigma\right) \tag{1}$$

Has many lovely properties, but fundamentally unsuitable for our work, as it assumes the distribution, rather than reconstructing it!

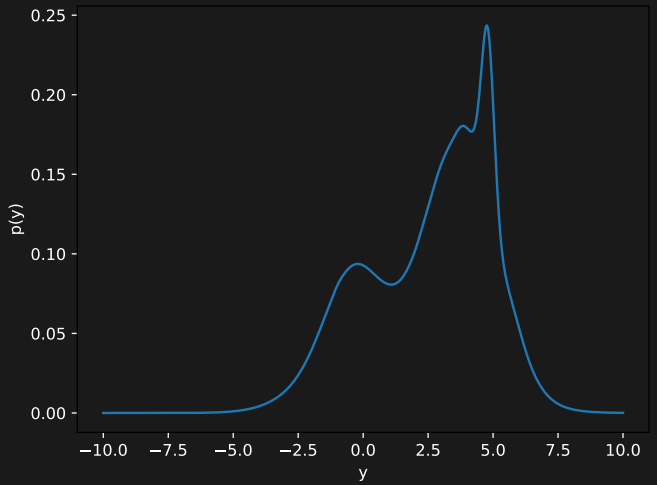
Partition Models

Split the covariate space up into *cells*



Partition Models

Each cell fits their own PDF:



Mixture of Experts

Instead of a hard partition, MoE probabilistically assign the 'problem' to an 'expert':

$$p(y|\boldsymbol{x}) = \sum_i w_i(\boldsymbol{x}) p_i \tag{2}$$

The w_i give different weights to the 'expert opinion' at different points in covariate space, coming to a different consensus.
Standard MoE uses Neural Networks to determine w_i (via complex partitioning of a learned hyperplane projection)

Models are not Emulators

Important!

The above models *are not emulators*. They provide ways to model the best-fit posterior distribution, conditioned on the observed data:

$$\text{Model}_{\boldsymbol{\Theta}}(\boldsymbol{x}) = p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\Theta})$$

What I desire is man's red fire
To make my dream come true

The Secret of Man's Red Fire

What we need it the *posterior predictive* distribution. The distribution conditioned *only* on the data:

The Secret of Man's Red Fire

What we need it the *posterior predictive* distribution. The distribution conditioned *only* on the data:

$$\text{Emulator}(\boldsymbol{x}) = p(\boldsymbol{y}|\boldsymbol{x}) = \int \text{Model}_{\boldsymbol{\Theta}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{\Theta} \tag{3}$$

The Secret of Man's Red Fire

What we need it the *posterior predictive* distribution. The distribution conditioned *only* on the data:

$$\text{Emulator}(\boldsymbol{x}) = p(\boldsymbol{y}|\boldsymbol{x}) = \int \text{Model}_{\boldsymbol{\Theta}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{\Theta} \tag{3}$$

An ape like me
Can learn to be human too

Our Requirements

We needed an emulator which can:

Our Requirements

We needed an emulator which can:

1. Reconstruct probability distribution function

Our Requirements

We needed an emulator which can:

1. Reconstruct probability distribution function
2. Allow both smooth and near-discontinuous changes in behaviour

Our Requirements

We needed an emulator which can:

1. Reconstruct probability distribution function
2. Allow both smooth and near-discontinuous changes in behaviour
3. Inherit 'expert knowledge' to reduce training requirements

Our Requirements

We needed an emulator which can:

1. Reconstruct probability distribution function
2. Allow both smooth and near-discontinuous changes in behaviour
3. Inherit 'expert knowledge' to reduce training requirements
4. Be interpretable and accessible

Department of Experts

A middle ground between partitioning and MoE models. We allow each expert to belong to a 'department' (a partition - which may have many experts within it), but also allow them to 'spend time' between departments.
The weight of an expert in the sum is then:

$$w_i(\boldsymbol{x}) = \sum_{\text{dep. } k} P(\boldsymbol{x} \in k) \times \frac{P(i \in k) P(i \text{ knows } \boldsymbol{x})}{\sum_j P(j \in k) P(j \text{ knows } \boldsymbol{x})} \tag{4}$$

These probabilities can be learned by the machine

The Clever Bit

This reduces trivially down into a standard MoE, with model parameters $\mathbf{\Theta}$

$$p(\boldsymbol{x}|\mathbf{\Theta}) = \sum_{w} w_i(\boldsymbol{x}|\Theta) p_i(\boldsymbol{x}|\mathbf{\Theta}) \tag{5}$$

The clever bit is that we allow each department to have their own 'grammar' that they use to talk to interpret the covariance space. The distance metric in each space is:

$$d_k(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(\boldsymbol{x} - \boldsymbol{y}) \cdot G_k(\boldsymbol{x} - \boldsymbol{y})} \tag{6}$$

Where G_k is a learned distance metric unique to each cell.

Why is this clever?

Why is this clever?

1. Allows w_i to encode highly nonlinear relationships in $\boldsymbol{x}$ *without resorting to black-box machinery*.

Why is this clever?

1. Allows w_i to encode highly nonlinear relationships in $\boldsymbol{x}$ *without resorting to black-box machinery*.
2. The metrics are easily interpreted as covariance measures localised within a department.

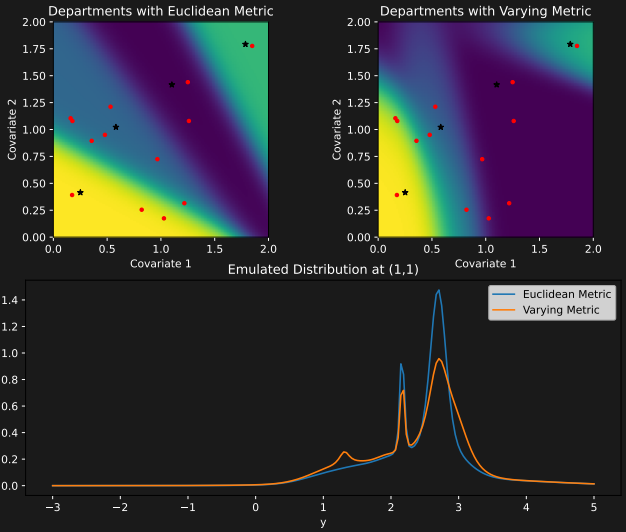
Why is this clever?

1. Allows w_i to encode highly nonlinear relationships in $\boldsymbol{x}$ *without resorting to black-box machinery*.
2. The metrics are easily interpreted as covariance measures localised within a department.
3. Gives the same predictive power as a high dimensional hidden-layer neural network

Why is this clever?

1. Allows w_i to encode highly nonlinear relationships in $\boldsymbol{x}$ *without resorting to black-box machinery*.
2. The metrics are easily interpreted as covariance measures localised within a department.
3. Gives the same predictive power as a high dimensional hidden-layer neural network
4. Constrained to produce 'interpretable results'.

A FADE Model


Departments with Euclidean Metric
Departments with Varying Metric
Covariate 1
Covariate 2
Emulated Distribution at (1,1)
Euclidean Metric
Varying Metric
p(y)
y